The Locality of Noncovalent Interactions: Machine-Learned Semilocal Corrections for DFT from Small Molecules

XX

(Dated: October 14, 2026)

In this article, we introduce a model structure and training loss for improving the accuracy of density functional theory (DFT) calculations using machine-learned semi-local corrections. The model is trained on the energy density of small molecules and then applied to larger systems. The results show that the learned corrections can significantly improve DFT accuracy for noncovalent interactions, even with a small training dataset whose largest molecules are C_2H_6, B_2H_6, and Si_2H_6. These findings demonstrate the transferability of semi-local corrections learned from small molecules to noncovalent interactions in larger systems.

I. INTRODUCTION

The well-known Kohn–Sham equation [1] is

$$\Big(-\frac{1}{2}\nabla^2 + v_{\text{ext}} + v_{\text{H}} + v_{\text{xc}}\Big)\phi_i = \epsilon_i\phi_i. \tag{1}$$

Here, ϕ_i is the Kohn–Sham orbital, ϵ_i is the corresponding orbital energy, v_{ext} is the external potential (for example, the electrostatic potential of the nuclei), v_{H} is the Hartree potential, and v_{xc} is the exchange-correlation potential. According to the Hohenberg-Kohn theorem [2], the exchange-correlation energy density e_{xc} is a unique functional of the ground-state electron density, $\rho = n_i|\phi_i|^2$, where n_i is the occupation number of the Kohn–Sham orbital. The Kohn–Sham equations are then solved iteratively using the potential term $v_{\text{xc}} = \delta E_{\text{xc}}/\delta\rho$. Once this functional is known, molecular properties can be computed accurately and efficiently. This is the central idea of density functional theory (DFT) [3].

One of the challenges in DFT is to determine the exact functional relationship between the exchange-correlation energy density e_{xc} and the electron density ρ. Numerous approximations have been proposed, including early local-density approximation functionals [3], hybrid functionals [4], and the rapidly expanding class of machine-learning (ML) functionals developed in recent years [5–9]. However, many ML functionals still suffer from limited transferability or require large training sets.

The most widely used benchmark for evaluating accuracy and transferability is GMTKN55. GMTKN55 is organized into five categories: reaction energies for small and large systems, reaction barrier heights, intermolecular non-covalent interactions (NCI), and intramolecular NCI. For many ML functionals, the training set is designed to have a category composition similar to that of the test set, which helps generalization to unseen data [7, 8]. In this article, we train only on one category, reaction energies for small systems, while testing on the full GMTKN55 dataset. This design is intended to test the transferability of an ML functional trained on a narrowly scoped category.

The remainder of this article is organized as follows. In Section 2, we introduce the neural network architecture used to learn the functional. In Section 3, we present the training procedure and results on GMTKN55. Finally, we conclude with a discussion of transferability.

II. THE NEURAL NETWORK

We use a Transformer [11] and an e3nn network [**?**] to learn the functional relationship between electron density and molecular energy density. After training, the model predicts molecular energy from electron density and is then embedded in an iterative self-consistent field (SCF) procedure to optimize the density.

The input set of energy densities $\tilde{\boldsymbol{e}}_{\text{cube}}(\mathbf{r})$ is defined as follows.

$$\begin{aligned}\tilde{\boldsymbol{e}}(\mathbf{r}_{\text{cube}}) =& \big\{\tilde{e}(\mathbf{r}'_c) \text{ for } \mathbf{r}'_c \in \mathbf{r}_{\text{cube}}\big\} \\ =& \big\{\tilde{e}(\mathbf{r}), \tilde{e}(\mathbf{r}_{\text{aux}})\big\}.\end{aligned} \tag{2}$$

The quantity $\tilde{e}(\mathbf{r})$ denotes the energy density computed from four components of the original B3LYP functional (LDA, VWN3, B88, and LYP) [4]. The set $\mathbf{r}_{\text{cube}}$ is a $3\times3\times3$ cube centered at grid point $\mathbf{r}$, with spacing d between adjacent points. Thus, each central grid point $\mathbf{r}$ is augmented with its local neighborhood to capture local structure and provide richer input features. We refer to all noncentral points in this cube as auxiliary grid points, $\mathbf{r}_{\text{aux}}$.

The energy density at the central grid point $\mathbf{r}$, denoted $e(\mathbf{r})$, is predicted by the network as

$$e_{\text{NN}}(\mathbf{r}) = \text{NN}\big(\tilde{\boldsymbol{e}}(\mathbf{r}_{\text{cube}}); \boldsymbol{\theta}\big), \tag{3}$$

where $\text{NN}\big(\tilde{\boldsymbol{e}}(\mathbf{r}_{\text{cube}}); \boldsymbol{\theta}\big)$ is the neural-network mapping parameterized by $\boldsymbol{\theta}$ and evaluated on $\tilde{\boldsymbol{e}}(\mathbf{r}_{\text{cube}})$. The total exchange-correlation energy is obtained by integrating the energy density over the entire grid. The exchange-correlation potential is then obtained as the functional derivative of this energy with respect to electron density and inserted into the Kohn–Sham equation Eq. (1), enabling molecular properties to be solved self-consistently. Notably, the predicted energy density at the central grid point depends only on the energy-density tensor of the local cube, $\tilde{\boldsymbol{e}}(\mathbf{r}_{\text{cube}})$; therefore, we classify this model as a semi-local ML functional.

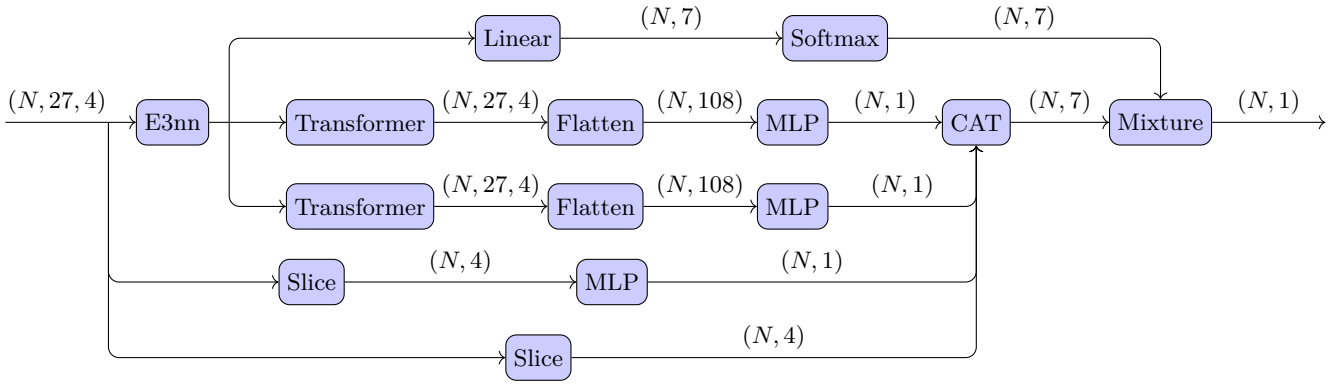


FIG. 1: The architecture of the neural network. The E3nn block [10] and the Slice block extract the invariant features from the input. The Mixture block combines the outputs from the different paths to produce the final prediction. The output of the neural network is the predicted energy density for the central grid point.

The weights of these auxiliary grid points, $\mathbf{r}_{\text{aux}}$, are set to zero, so they do not affect the number of electrons or the total energy, they only serve as additional features for the neural network. Auxiliary grid points do contribute to the exchange-correlation potential, which is calculated using the weights of the central grid point and integrated over the entire grid. Details of the formulation for exchange-correlation potential are provided in the Appendix.

The network architecture is shown in Fig. 1. The model contains approximately 1.2 million parameters. The model input is the energy-density tensor $\tilde{\boldsymbol{e}}_{\text{cube}}(\mathbf{r})$ of size $(N, 27, 4)$, and the output is the energy density $e(\mathbf{r})$ of size $(N, 1)$. Here, N denotes the number of grid points in a standard DFT calculation. We use a combination of E3nn, Transformer, and Mixture blocks to process the input features. The Mixture block combines outputs from these pathways to produce the final prediction, which is similar to the Mixture of Experts (MoE) architecture [12]. We use an E3nn block [10] together with a Slice block to extract invariant features from $\tilde{\boldsymbol{e}}(\mathbf{r}_{\text{cube}})$. The Transformer block consists of multiple layers built from the scaled dot-product attention component of the original Transformer [11]. We use GELU consistently in the scalar nonlinear layers as a smooth baseline activation. A radial-MLP comparison with SiLU showed comparable validation performance, providing no clear motivation to change the baseline. Additional architectural details are provided in the GitHub repository[**?**].

III. DATASET AND THE LOSS FUNCTION

A. Dataset and the loss function

The dataset we use to train the neural network is listed in the Appendix. We use 128 molecules which consistent of less than 1-2 heavy (non-hydrogen) atoms as the training set and 34 molecules contain 4-5 heavy atoms as the validation set. In this article, we use the CCSD(T)/Def2-QZVPPD [13–15] to calculate the electron density and the energy density of the molecules.

The loss function is defined as

$$\begin{aligned} L_e =& N_{\text{atom}} \frac{\left(\int \mathrm{d}\mathbf{r}_i \left(\bar{e}(\mathbf{r}_i) - e(\mathbf{r}_i)\right)\right)^2}{\left|\int \mathrm{d}\mathbf{r}_i \bar{e}(\mathbf{r}_i)\right| + \epsilon} + N_{\text{atom}} \frac{\left(\bar{E}^{\text{AE}} - E^{\text{AE}}\right)^2}{\left|\bar{E}^{\text{AE}}\right| + \epsilon} \\ &+ N_{\text{atom}} \lambda_{\text{ABS}} \int \mathrm{d}\mathbf{r}_i \left(\bar{e}(\mathbf{r}_i) - e(\mathbf{r}_i)\right)^2 + N_{\text{atom}} \left(\bar{F} - F\right)^2. \end{aligned} \tag{4}$$

where N_{atom} is the number of atoms in the molecule, E^{AE} and $\bar{E}^{\text{AE}}$ are the predicted and target atomic energies, and F and $\bar{F}$ are the predicted and target forces, respectively. The parameter ϵ is a small constant introduced to avoid division by zero and control the scale of the loss; in this work, we set $\epsilon = 1 \times 10^{-4}$. The first term represents the relative error in the total energy, the second term captures the atomic-energy error, the third term penalizes the absolute energy-density error, and the fourth term optimizes the force. The optimization of the force will stabilize the SCF procedure in the testing phase. See the Appendix for the definitions of atomic energy and force.

Model	Model 1	Model 2	B3LYP	
Subset1				
Subset2				
Subset3				
Subset4				
Subset5				
Total				

B. Training details

The neural network is trained using the AdamW optimizer [16] with the scheduler of cosine annealing with warm restarts [17]. We set the initial learning rate to be 1×10^{-4} and then gradually decrease it to 0 using the cosine annealing scheduler, with a restart every 160 epochs and a multiplication factor of 2 for the initial learning rate after each restart. The parameters of the AdamW optimizer are set to be $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\epsilon = 1 \times 10^{-8}$, and the weight decay is set to be 1×10^{-12}. The weights in the loss function Eq. (**??**) are set to be $\lambda_{\text{ABS}} = 0.01$. We use a batch size of 1 and train the model for 10000 epochs, the training process is last for around 20 days. The training process is monitored using the validation set every 5 epochs, and the model with the lowest validation loss is selected as the final model.

The detail of the training dataset and validation dataset is listed in the Appendix. The training dataset consists of 69 molecules (set A and set B) or 128 molecules (set A, set B and set C) with $1-2$ heavy (non-hydrogen) atoms, while the validation dataset consists of 115 molecules with $4-5$ heavy atoms. The validation loss is calculated using only the first two items in the loss function Eq. (4), which are the relative error in the total energy and the atomic energy error, to avoid the difficulty of calculating the absolute error and force in the total energy for large molecules.

The training is performed on a single NVIDIA A100 GPU with 80 GB memory with double precision training using PyTorch [18] and the training takes around 10 days to complete. The version of PyTorch used is 2.8.0+cu128, and CUDA version is 12.8, the version of driver is 535.247.01.

IV. RESULTS

A. Testing on the GMTKN55 Dataset

We use the original GMTKN55 dataset and WTMAD-2 metric for evaluating the performance of our model. The first one is the model trained on set A and set B, while the second one is the model trained on set A, set B, and set C. Others density functional approximation (DFA) can be found in the original GMTKN55 dataset.

B. Comparison with other machine learning functionals

The main difference between our method and other machine learning functionals [5–9] is that we use the energy density as the target of the neural network instead of the total energy.

[1] W. Kohn and L. J. Sham, Self-Consistent Equations Including Exchange and Correlation Effects, Phys. Rev. **140**, A1133 (1965).

[2] P. Hohenberg and W. Kohn, Inhomogeneous Electron Gas, Phys. Rev. **136**, B864 (1964).

[3] R. G. Parr, Density Functional Theory of Atoms and Molecules, in *Horiz. Quantum Chem.*, edited by K. Fukui and B. Pullman (Springer Netherlands, Dordrecht, 1980) pp. 5–15.

[4] J. P. Perdew, J. A. Chevary, S. H. Vosko, K. A. Jackson, M. R. Pederson, D. J. Singh, and C. Fiolhais, Atoms, molecules, solids, and surfaces: Applications of the generalized gradient approximation for exchange and correlation, Phys. Rev. B **46**, 6671 (1992).

[5] R. Nagai, R. Akashi, and O. Sugino, Completing density functional theory by machine learning hidden messages from molecules, Npj Comput. Mater. **6**, 43 (2020).

[6] Y. Chen, L. Zhang, H. Wang, and W. E, DeePKS: A Comprehensive Data-Driven Approach toward Chemically Accurate Density Functional Theory, J. Chem. Theory Comput. **17**, 170 (2021).

[7] J. Kirkpatrick, B. McMorrow, D. H. P. Turban, A. L. Gaunt, J. S. Spencer, A. G. D. G. Matthews, A. Obika, L. Thiry, M. Fortunato, D. Pfau, L. R. Castellanos, S. Petersen, A. W. R. Nelson, P. Kohli, P. Mori-Sánchez, D. Hassabis, and A. J. Cohen, Pushing the frontiers of density functionals by solving the fractional electron problem, Science **374**, 1385 (2021).

[8] G. Luise, C.-W. Huang, T. Vogels, D. P. Kooi, S. Ehlert, S. Lanius, K. J. H. Giesbertz, A. Karton, D. Gunceler, M. Stanley, W. P. Bruinsma, L. Huang, X. Wei, J. G. Torres, A. Katbashev, R. C. Zavaleta, B. Máté, S.-O. Kaba, R. Sordillo, Y. Chen, D. B. Williams-Young, C. M. Bishop, J. Hermann, R. van den Berg, and P. Gori-Giorgi, Accurate and scalable exchange-correlation with deep learning (2025), arXiv:2506.14665 [physics].

[9] Z. An, J. Wang, Y. Zhang, Z. Li, J. Wu, Y. Zheng, G. Chen, and X. Zheng, Mitigating error cancellation in density functional approximations via machine learning correction, J. Chem. Phys. **163**, 054111 (2025).

[10] M. Geiger and T. Smidt, E3nn: Euclidean Neural Networks (2022), arXiv:2207.09453 [cs.LG].

[11] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, Attention Is All You Need (2023), arXiv:1706.03762 [cs].

[12] W. Cai, J. Jiang, F. Wang, J. Tang, S. Kim, and J. Huang, A Survey on Mixture of Experts in Large Language Models, IEEE Trans. Knowl. Data Eng. , 1 (2025),

arXiv:2407.06204 [cs.LG].
[13] R. J. Bartlett and M. Musiał, Coupled-cluster theory in quantum chemistry, Rev. Mod. Phys. **79**, 291 (2007).
[14] F. Weigend and R. Ahlrichs, Balanced basis sets of split valence, triple zeta valence and quadruple zeta valence quality for H to Rn: Design and assessment of accuracy, Phys. Chem. Chem. Phys. **7**, 3297 (2005).
[15] J. Zheng, X. Xu, and D. G. Truhlar, Minimally augmented Karlsruhe basis sets, Theor. Chem. Acc. **128**, 295 (2011).
[16] I. Loshchilov and F. Hutter, Decoupled Weight Decay Regularization (2019), arXiv:1711.05101 [cs].
[17] I. Loshchilov and F. Hutter, SGDR: Stochastic Gradient Descent with Warm Restarts (2017), arXiv:1608.03983 [cs].
[18] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, PyTorch: An Imperative Style, High-Performance Deep Learning Library (2019), arXiv:1912.01703 [cs].
[19] L. Goerigk, A. Hansen, C. Bauer, S. Ehrlich, A. Najibi, and S. Grimme, A look at the density functional theory zoo with the advanced GMTKN55 database for general main group thermochemistry, kinetics and noncovalent interactions, Phys. Chem. Chem. Phys. **19**, 32184 (2017).

Appendix A: Training Details

1. Dataset

The dataset we use are listed in Table I, all the Datasets are obtained from the GMTKN55 datasets [19]. We use the molecules in datasets A, B, and C as the training set and the molecules in datasets A′ and B′ as the validation set. Datasets A, B, and C are defined as follows: dataset A comprises single-atom systems from GMTKN55 [19]; dataset B comprises molecules with one heavy (non-hydrogen) atom from W4-11; and dataset C comprises molecules with two heavy atoms from W4-11. This lead to 47 atoms/ions and 81 molecules in the training set and 34 molecules in the validation set, while the number of molecules in GMTKN55 datasets is 2462 [19]. To improve training efficiency and balance the dataset across different atom species, we add 9 additional copies of dataset A.

In this article, we use the electron density and energy density from CCSD(T)/Def2-QZVPPD [13–15] as the input and target of the molecules. This leads to a significant label error, which is estimated to be 2.33 kcal/mol as the mean absolute error (MAE) for the W4-11 dataset. Then all the test procedures are performed with the a smaller basis set, Def2-QZVP(D) [14, 15], which means the Def2-QZVPD basis set is used for anions and the Def2-QZVP basis set is used for other molecules, to reduce the computational cost.

We use the energy density calculated by the four functionals (LDA, VWN3, B88, and LYP) from the original B3LYP functional [4] as the input features of the neural network, not the electron density and the derivatives of the electron density directly.

$$\tilde{e}(\mathbf{r}) = \big(e_{\mathrm{LDA}}(\mathbf{r}), e_{\mathrm{VWN3}}(\mathbf{r}), e_{\mathrm{B88}}(\mathbf{r}), e_{\mathrm{LYP}}(\mathbf{r})\big). \tag{A1}$$

This choice is motivated by the very wide dynamic range of the electron density, ρ, and its derivatives, which complicates learning and can destabilize both the optimization and the validation process. By using the energy densities from established components (LDA, VWN3, B88, LYP) as input features, we provide numerically stable, physically informed representations of the local electronic environment.

The network outputs the residual energy density, defined as $e(\mathbf{r}) = e_{\mathrm{xc}}^{\mathrm{CCSD(T)}}(\mathbf{r}) - e_{\mathrm{xc}}^{\mathrm{B3LYP}}(\mathbf{r})$, where the B3LYP term is computed from the input features and the CCSD(T) term serves as the target. The total exchange-correlation energy density is then reconstructed by adding the B3LYP energy density back to the predicted residual, $E_{\mathrm{xc}}^{\mathrm{pred}} = E'_{\mathrm{xc}} + E_{\mathrm{xc}}^{\mathrm{B3LYP}}$.

Appendix B: Auxiliary Grid Points

To provide additional features for the neural network, we consider a small cube surrounding each grid point $\mathbf{r}_i = (x_i, y_i, z_i)$. This cube has dimensions of $3 \times 3 \times 3$, with d representing the distance between two adjacent grid points. To make it clear, the auxiliary grid points $\mathbf{r}_{i;\mathrm{aux}}$ are defined as all grid points within this cube, excluding the central point $\mathbf{r}_i$ itself.

$$\mathbf{r}_{i;\mathrm{aux}} = \big\{\mathbf{r}'_i \in \mathbf{r}_{i;\mathrm{cube}} \text{ and } \mathbf{r}'_i \neq \mathbf{r}_i\big\}. \tag{B1}$$

In the standard DFT procedure, each grid point is assigned a weight w_i, which is used to perform numerical integration. We set all the weights of the auxiliary grid points ($w_{i;\mathrm{aux}}$) to be zero. Thus, they do not contribute to the total energy in the DFT procedure.

$$E'_{\mathrm{xc}} = \int e_{\mathrm{NN}}(\mathbf{r})\mathrm{d}\mathbf{r} = \int \mathrm{dNN}\big(\tilde{\boldsymbol{e}}(\mathbf{r}_{\mathrm{cube}}); \boldsymbol{\theta}\big). \tag{B2}$$

The contribution of the electron density ρ to the $\mu\nu$-th element of Fock matrix which contributed by the ML exchange-correlation, $\mathrm{NN}\big(\tilde{\boldsymbol{e}}(\mathbf{r}_{\mathrm{cube}}); \boldsymbol{\theta}\big)$, is calculated as

$$\begin{aligned} F_{\mu\nu}^{\mathrm{ML}} =& \frac{\delta E'_{\mathrm{xc}}}{\delta P_{\mu\nu}} \\ =& \int \mathrm{d}\mathbf{r} \sum_{\mathbf{r}' \in \mathbf{r}_{\mathrm{cube}}} \frac{\delta e_{\mathrm{NN}}(\mathbf{r}')}{\delta \rho(\mathbf{r}')} \chi_\mu(\mathbf{r}')\chi_\nu(\mathbf{r}') \\ =& \int \mathrm{d}\mathbf{r} \sum_{\mathbf{r}' \in \mathbf{r}_{\mathrm{cube}}} \frac{\delta \mathrm{NN}\big(\tilde{\boldsymbol{e}}(\mathbf{r}'); \boldsymbol{\theta}\big)}{\delta \rho(\mathbf{r}')} \chi_\mu(\mathbf{r}')\chi_\nu(\mathbf{r}') \\ =& \int \mathrm{d}\mathbf{r} \sum_{\mathbf{r}' \in \mathbf{r}_{\mathrm{cube}}} \frac{\partial \mathrm{NN}\big(\tilde{\boldsymbol{e}}(\mathbf{r}'); \boldsymbol{\theta}\big)}{\partial \tilde{e}(\mathbf{r}')} \frac{\delta \tilde{e}(\mathbf{r}')}{\delta \rho(\mathbf{r}')} \chi_\mu(\mathbf{r}')\chi_\nu(\mathbf{r}'). \end{aligned} \tag{B3}$$

and the contribution of the derivative of the electron density $\nabla\rho$ to the Fock matrix is calculated similarly. Consequently, the auxiliary grid points influence the potential just like the center points with the same weights, w_i. This ensures the procedure remains consistent with the standard DFT framework with only minor modifications.

To make the neural network can capture more information about the local environment of the grid point $\mathbf{r}_i$ in the training set, we set the cube to be aligned with the main rotation axis of the secondary derivative matrix of the electron density at the grid point $\mathbf{r}_i$. This secondary derivative matrix is defined as

$$H(\mathbf{r}_i) = \begin{pmatrix} \frac{\partial^2\rho(\mathbf{r}_i)}{\partial x^2} & \frac{\partial^2\rho(\mathbf{r}_i)}{\partial x\partial y} & \frac{\partial^2\rho(\mathbf{r}_i)}{\partial x\partial z} \\ \frac{\partial^2\rho(\mathbf{r}_i)}{\partial y\partial x} & \frac{\partial^2\rho(\mathbf{r}_i)}{\partial y^2} & \frac{\partial^2\rho(\mathbf{r}_i)}{\partial y\partial z} \\ \frac{\partial^2\rho(\mathbf{r}_i)}{\partial z\partial x} & \frac{\partial^2\rho(\mathbf{r}_i)}{\partial z\partial y} & \frac{\partial^2\rho(\mathbf{r}_i)}{\partial z^2} \end{pmatrix}. \tag{B4}$$

The eigenvectors $\mathbf{u}(\mathbf{r}_i)$ of this matrix represent the main rotation axes, and we align the cube with these axes.

$$\begin{aligned} \mathbf{r}_{i;\mathrm{cube}} =& \big\{\mathbf{r}_i + d(m\mathbf{u}_1(\mathbf{r}_i) + n\mathbf{u}_2(\mathbf{r}_i) + p\mathbf{u}_3(\mathbf{r}_i)) \\ & \text{for } m, n, p \in \{-1, 0, 1\}\big\}. \end{aligned} \tag{B5}$$

Although this alignment process adds some computational cost, it provides a much more local reference frame

TABLE I: The training and validation dataset.

	Original Dataset	Size	Number of heavy (non-hydrogen) atoms	Number of hydrogen atoms
A	GMTKN55 [1]	47	1	0
B	W4-11	22	1	$\neq 0$
C	W4-11	59	2	–
A′	GMTKN55	23	5	≤ 3
B′	GMTKN55	11	6	≤ 3

[1] Not include all atoms-species in the GMTKN55 dataset, such as Se, Ge, As, Te, I, Bi, Pb, and Sb.

for the input features, which may help the model learn more effectively. While in the test set, we do not perform this alignment process, and the cube is aligned with the global coordinate system.

Appendix C: Ablation Study